

ETC4500/ETC5450

Advanced R programming

Week 10: Metaprogramming



Outline

- 1 Metaprogramming
- 2 (Non-)standard evaluation
- 3 Tidy evaluation
- 4 Software design

Outline

- 1 Metaprogramming
- 2 (Non-)standard evaluation
- 3 Tidy evaluation
- 4 Software design

Metaprogramming

Metaprogramming is *programming* about programming.

In other words, writing code that can *inspect* and *modify* code.

Metaprogramming

Metaprogramming is *programming* about programming.

In other words, writing code that can *inspect* and *modify* code.

💡 A powerful idea

Unlike most programming languages, R *embraces* metaprogramming and non-standard evaluation (NSE).

This powers much of the strange but wonderful interface designs in R and its packages.

The rlang package

```
library(rlang)
```



A package for writing R code that interacts with R code.

! Not a new idea!

Metaprogramming/NSE doesn't require the rlang package.

There are base R equivalents to the functions shown today.

NSE is widely used in base R (not just in the tidyverse!)

Parsing code

Every time you run code anywhere in R it needs to be ‘interpreted’ by the parser.

The parser reads unstructured text (your written code) and interprets it as an expression.

```
# parse(text = "seq(1, 10, by = 0.5)")  
parse_expr("seq(1, 10, by = 0.5)")
```

```
seq(1, 10, by = 0.5)
```

Code is data

Expressions (code) can be used like any other data in R.

```
my_seq <- parse_expr("seq(1, 10, by = 0.5)")  
my_seq
```

```
seq(1, 10, by = 0.5)
```

```
class(my_seq)
```

```
[1] "call"
```

Code is data

Expressions (code) can be used like any other data in R.

```
my_seq <- parse_expr("seq(1, 10, by = 0.5)")  
my_seq
```

```
seq(1, 10, by = 0.5)
```

```
eval(my_seq)
```

```
[1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5 6.0 6.5 7.0  
7.5  
[15] 8.0 8.5 9.0 9.5 10.0
```

Outline

- 1 Metaprogramming
- 2 (Non-)standard evaluation
- 3 Tidy evaluation
- 4 Software design

Code evaluation

Standard evaluation

- The code and environment is unchanged.
- The result is evaluated as expected.

Code evaluation

Standard evaluation

- The code and environment is unchanged.
- The result is evaluated as expected.

Non-standard evaluation (NSE)

- The code and/or the environment is changed.
- Leading to the evaluated result changing.

Standard or non-standard evaluation?

Question!

Do the following functions use standard evaluation or NSE?

- `library(rlang)`
- `a + b * c`
- `mtcars |> select(cyl)`
- `read_csv("data/study.csv")`
- `ggplot() + geom_line()`
- `mtcars |> mutate(wt/hp)`
- `with(mtcars, wt/hp)`

The building blocks of code evaluation

There are four building blocks used in evaluating code.

- **Constants:** A specific value like `1` or `"data/study.csv"`.
- **Symbols:** A name of an object, like `pi` or `mtcars`.
- **Expressions:** Code structured as an “Abstract Syntax Tree” (AST) of calls and symbols.
- **Environments:** The place where named objects are found.

The building blocks of code evaluation

There are four building blocks used in evaluating code.

- **Constants:** A specific value like `1` or `"data/study.csv"`.
- **Symbols:** A name of an object, like `pi` or `mtcars`.
- **Expressions:** Code structured as an “Abstract Syntax Tree” (AST) of calls and symbols.
- **Environments:** The place where named objects are found.

 Question?

How are these building blocks used together to construct and evaluate code?

The building blocks of code evaluation

In `rlang`, we have three main building block functions:

- `sym("pi")`: a symbol/name like `pi`
- `expr(1/pi)`: an expression for `1/pi`
- `quo(1/pi)`: a *quosure* (expression **and** environment)

The building blocks of code evaluation

 Your turn!

Spot the difference.

How do the results of the following functions differ?

- `sym("2 * pi")`
- `expr(2 * pi)`
- `quo(2 * pi)`

Capturing code

More often than not, NSE involves capturing user code that was used in your function. This is done with `en*()` functions:

- `ensym(x)`: capture a symbol
- `enexpr(x)`: capture an expression
- `enquo(x)`: capture a quosure

Capturing code

More often than not, NSE involves capturing user code that was used in your function. This is done with `en*()` functions:

- `ensym(x)`: capture a symbol
- `enexpr(x)`: capture an expression
- `enquo(x)`: capture a quosure

These must be used inside functions, for example:

```
capture_expr <- function(x) {  
  enexpr(x)  
}  
capture_expr(1/pi)
```

Capturing code

 Your turn!

What is the difference between these two functions?

```
f <- function(x) x  
g <- function(x) enexpr(x)
```

Try calling `f(1 + 2)` and `g(1 + 2)`. What does each return?

Capturing code

Your turn!

Write a function `assert_positive` that evaluates its argument and stops with an informative error if the result is not positive. The error message should include the original expression.

```
assert_positive(sqrt(4))    # returns 2
assert_positive(log(-1))    # error: "log(-1) must be positive"
```

Hint: use `enexpr()` to capture the expression and `deparse()` to turn it into a string.

Unquoting (bang-bang!!)

Why doesn't the following code work?

```
log_expr <- function(x) {  
  # Capture expression  
  x <- enexpr(x)  
  # Return new expression with log()  
  expr(log(x))  
}  
log_expr(1/pi)
```

```
log(x)
```

Unquoting (bang-bang!!)

To use captured code in our functions, we need to unquote it.

```
log_expr <- function(x) {  
  # Capture expression  
  x <- enexpr(x)  
  # Return new expression with log()  
  expr(log (!!x))  
}  
log_expr(1/pi)
```

```
log(1/pi)
```

`expr(log (!!x))` will create an expression (`expr()`) that replaces `x` with its value (`1/pi`).

Unquoting (bang-bang!!)

Suppose we wanted to programmatically `filter()` `mtcars$cyl`:

```
cyl <- 4  
mtcars |>  
  filter(cyl == cyl)
```

Question

What's the problem? How can unquoting help?

Unquoting (bang-bang!!)

Inside a function, the column name comes from the *caller* – so we must capture it first with `enquo()`, then unquote it with `!!`.

```
filter_col <- function(data, col, val) {  
  col <- enquo(col)  
  data |> filter(!!col == val)  
}  
filter_col(mtcars, cyl, 4)
```

Unquoting (bang-bang!!)

Inside a function, the column name comes from the *caller* – so we must capture it first with `enquo()`, then unquote it with `!!`.

```
filter_col <- function(data, col, val) {  
  col <- enquo(col)  
  data |> filter(!!col == val)  
}  
filter_col(mtcars, cyl, 4)
```

The pattern `enquo()` then `!!` appears so often that it has a dedicated shorthand.

Embracing inputs ({{curly-curly}})

A shorthand for `!!enquo(x)` is known as ‘embrace’ or ‘curly-curly’. The code `{{x}}` is identical to `!!enquo(x)`.

```
filter_col <- function(data, col, val) {  
  data |> filter({{ col }} == val)  
}  
filter_col(mtcars, cyl, 4)
```

Embracing inputs ({{curly-curly}})

Consider this function for summarising a value's range:

```
var_summary <- function(data, var) {  
  data |>  
    summarise(n = n(), min = min({{ var }}), max = max({{ var }}))  
}  
mtcars |>  
  group_by(cyl) |>  
  var_summary(mpg)
```

Question

Why is `enquo()` important here?

Capturing multiple expressions

To capture multiple expressions from `...`, use `enquos(...)`. This returns a named list of quosures.

```
print_conds <- function(...) {  
  conds <- enquos(...)  
  purrr::walk(conds, \(cond) cat(as_label(cond), "\n"))  
}  
print_conds(cyl == 4, wt < 2.5, am == 1)
```

Capturing multiple expressions

To capture multiple expressions from ..., use `enquos(...)`. This returns a named list of quosures.

```
print_conds <- function(...) {  
  conds <- enquos(...)  
  purrr::walk(conds, \(cond) cat(as_label(cond), "\n"))  
}  
print_conds(cyl == 4, wt < 2.5, am == 1)
```



`as_label()` turns a quosure or expression into a string, which is useful for printing error messages.

Outline

- 1 Metaprogramming
- 2 (Non-)standard evaluation
- 3 Tidy evaluation
- 4 Software design

Tidy evaluation

Tidy evaluation searches the variables of the data first, followed by the search path of the user's environment.

This is a type of NSE, since it changes the environment in which code is ran.

```
mtcars |>  
  mutate(mpg/wt)
```

`mpg/wt` would ordinarily error since `mpg` and `wt` aren't found, but `mutate()` uses NSE to first search the dataset.

Tidy evaluation

This is accomplished using `eval_tidy()`, with the arguments:

- `expr`: The expression (code) to evaluate
- `data`: The dataset 'mask' to search first
- `env`: The environment to search next.

Unlike `eval()`, this will:

- Respect the environments of quosures
- Attach *pronouns* for `.data` and `.env`

Tidy evaluation

We can use `eval_tidy()` to create a simple `dplyr::mutate()` function variant.

```
my_mutate <- function(.data, mutation) {  
  mutation <- enquo(mutation)  
  result <- eval_tidy(mutation, data = .data, env = caller_env())  
  .data[[as_label(mutation)]] <- result  
  .data  
}  
mtcars |>  
  my_mutate(mpg/wt)
```

Tidy evaluation

We can use `eval_tidy()` to create a simple `dplyr::mutate()` function variant.

```
my_mutate <- function(.data, mutation) {  
  mutation <- enquo(mutation)  
  result <- eval_tidy(mutation, data = .data, env = caller_env())  
  .data[[as_label(mutation)]] <- result  
  .data  
}  
mtcars |>  
  my_mutate(mpg/wt)
```

Question

What features are missing in our function compared to `dplyr::mutate()`?

Data masks

`eval_tidy()` can use **any named list** as a data mask:

```
name  <- c("Alice", "Bob", "Carol")
score <- c(82, 95, 71)
mask  <- list(name = name, score = score)
eval_tidy(quo(score > 80), data = mask)
```

```
[1] TRUE TRUE FALSE
```

Data masks

`eval_tidy()` can use **any named list** as a data mask:

```
name <- c("Alice", "Bob", "Carol")
score <- c(82, 95, 71)
mask <- list(name = name, score = score)
eval_tidy(quo(score > 80), data = mask)
```

```
[1] TRUE TRUE FALSE
```

This means you can evaluate filter expressions against fields of any custom object, as long as you construct the mask yourself.

Data masks

Quosures also capture the *caller's environment*, so expressions can reference variables that aren't in the mask.

```
threshold <- 80  
f <- function(cond) eval_tidy(enquo(cond), data = mask)  
f(score > threshold) # threshold found in caller's env, not the mask
```

```
[1] TRUE TRUE FALSE
```

Your turn!

Your turn!

Build a mask from the first 6 rows of `airquality` and evaluate the expression `Temp > 70` against it.

```
mask <- as.list(head(airquality))  
eval_tidy(quo(Temp > 70), data = mask)
```

Which rows match? What happens if you try `Temperature > 70` instead?

Filtering with multiple conditions

Capture multiple filter expressions from ..., evaluate each against the mask, then AND them together.

```
my_filter <- function(.data, ...) {  
  conds <- enquos(...)  
  mask <- as.list(.data)  
  keep <- purrr::map(conds, \(cond) eval_tidy(cond, data = mask))  
  keep <- Reduce("&", keep)  
  .data[keep, ]  
}  
my_filter(mtcars, cyl == 4, wt < 2.5)
```

Your turn!

Your turn!

Modify `my_filter` to accept an extra argument `.op = "all"` that controls how multiple conditions are combined.

- `.op = "all"` (default): keep rows matching **every** condition
- `.op = "any"`: keep rows matching **at least one** condition

```
my_filter(airquality, Temp > 90, Month == 8, .op = "any")
```

Hint: look at what `Reduce("&", keep)` is doing, and think about what to change.

Outline

- 1 Metaprogramming
- 2 (Non-)standard evaluation
- 3 Tidy evaluation
- 4 Software design

Software design

NSE is a powerful tool for software design.

But once again, with great power...

... comes great responsibility!

Tidyverse design principles

Notice how little NSE the tidyverse uses to great effect.

A lot of thought has gone into designing the tidyverse, which mostly uses standard evaluation: <https://design.tidyverse.org/>

Tidyverse design principles

Notice how little NSE the tidyverse uses to great effect.

A lot of thought has gone into designing the tidyverse, which mostly uses standard evaluation: <https://design.tidyverse.org/>

A design compromise

While very appreciated by users, NSE introduces a lot of complexity when programming with tidyverse packages.

Software design with NSE

In most cases you shouldn't add NSE to your package.

Software design with NSE

In most cases you shouldn't add NSE to your package.

! Why?

NSE can be incredibly confusing for users!

Code might work outside your function, but be completely different when used inside it.

Software design with NSE

In most cases you shouldn't add NSE to your package.

! Why?

NSE can be incredibly confusing for users!

Code might work outside your function, but be completely different when used inside it.

Understanding NSE however is very useful for advanced use of tidyverse packages in non-interactive contexts.

Software design with NSE

If you must use NSE, you should:

- Use it sparingly
- Be consistent
- Clearly document it
- Get a lot of design benefit from it
(not just for slightly less typing!)